Lecture 1 - Chapter 1

June

Programming Languages:

Machine Language: A set of primitives instructions build into every computer

Assembly Language: Developed to make programming easy: An assembler is used to convert assembly language programs into machine code.

High-Level Language: English-like and easy to learn & program.

Terminology:

Methods: Contains Statements that performs a task.

Java: public static void main (String[] args)

Blocks: Code that is between an opening curly brace and closing curly brace { }

Statements: Represents actions. Must end with a Semicolon ;

Source File: A file with the .java extension

String: Also called a character string. A bunch of Characters between double qoutes "This is a String"

Reserved words (keywords): int, double, main

Identifiers: Basically a name chosen by the programmer for Variables, constants, methods.

Programming Errors:

Syntax errors: Detected by the Compiler. If you leave off a Semicolon, have a typo in your code, leave off a curly brace, ect.

Runtime Error: Causes the program to abort. Cause the code to crash - will see error message in console window in Eclipse. If you attempt to divide by 0 your code will have a runtime error

Logic Errors: Produces incorrect result. Occurs when your code does not produce the expected result

Lecture 2 - Chapter 1

Terminology:

Methods: Contains statements that perform a task.

Blocks: Code that is between an opening curly brace and closing curly brace.

Statements: Represent action. Must end with a Semicolon

Source File: A file with the .java extension.

String: Also called a character String. A String is a bunch of characters enclosed between double quotes- "This is a String"

Reserved Words (Keywords): Words that have a specific meaning in the language and cannot be used for other purposes. int, double, main

Identifer: Basically a name chosen by the programmer for variables, Constants, methods.

Lecture 3- Chapter 2

June

Notes:

Modulus = Remainder

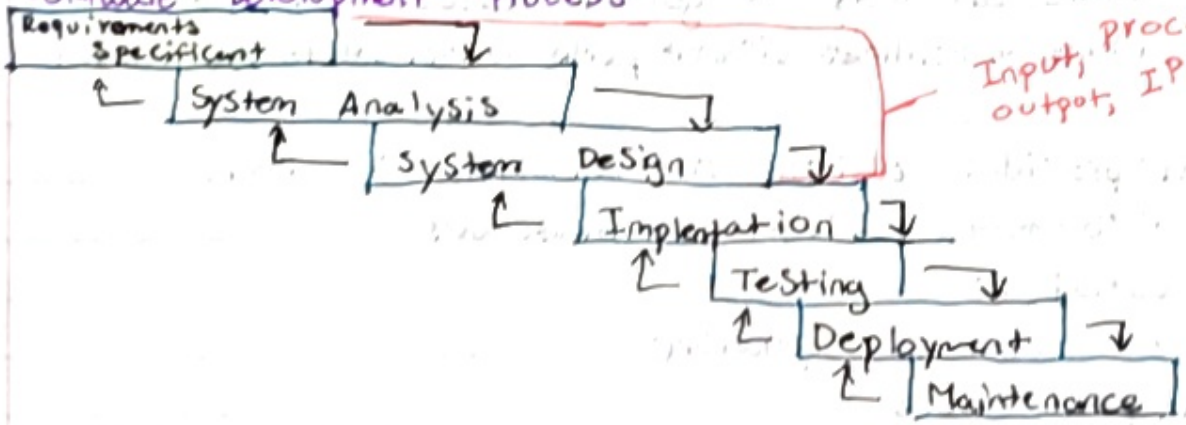
JShell is a command used in Java 9

Time: System.currentTimeMillis();

Increment and Decrement Operators:

++ var	preincrement
var ++	postincrement
-- var	predecrement
var --	postdecrement

Software Development Process



pg 51-58

Requirement Specificant: Formal process that seeks to understand the problem & document in detail what the software system needs to do.

System Analysis: Seeks to analyze the buisness process in terms of data flow, and to identify the system's input and output.

System Design: The process of designing the system's components.

Implentation: The process of translating the system design into programs. Separate programs are written for each component and put to work together.

Testing: Ensures that the code meets the requirements specification and weeds out bugs.

Deployment: Makes the project available for use.

Maintance: Maintance is concerned with changing & improving the product.

Lecture 4 - Bay Chapter 2

Terminology:

Boolean data type: used to store the values - True or false

Relational operation: < <= == != > >=

Logical operations: ! = Not && and || or ^ exclusive or

A	B	&&	||	^
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	error

Assignment Statement: Way to get a value in a variable

Selection Statement: Allows different paths within code to be taken

Notes:

if Statement: provides ability to Selection if then.

One-way if-Statement: if-else Statements

Scope of Variables:

Fitting into order of operating:

Anything in parantheses

expr ++ expr-- (postfix)

++expr --expr (prefix)

! (not)

* / % (multi, divid, remainder)

+ - (add, sub)

&& (and)

|| (or)

== != (equality)

< <= > >= (relation operators)

= += -= *= /= %= (assignment, augment assign.)

Truth table: (&&)

P_1	P_2	P_1 && P_2
F	F	F
F	T	F
T	F	F
T	T	T

P_1	P_2	P_1 && P_2
T	F	F
T	T	T

Truth Table (!):

P	!p	example
T	F	!(age > 18) is false
F	T	!(age > 18) is True

Truth table (||):

P_1	P_2	$P_1 \mid\mid P_2$
F	F	F
F	T	T
T	F	T
T	T	T

Swith Statement:

Swith Statement: Like an if-Statement, the Swith provides ability to select a path of execution from a group of possibility

```
Swith (Statis) {
   Case 0: ← break;
   Case 1:
   default:
}
```

Rules: Must always be in parenthesis. Must evalute to a value of type char, String, byte. Short, int, long. The data type of the expression must math data type of value 1, value 2, ect. The break Statement is optional and ends a case.

Which One Should Be used - if or Switch:

Use an If Statement: Need to test for boolean conditions.

Use an Switch Statement: Situations where looking for a Constant value and the cases are Simple.

Conditional Expressions:

Purpose: Shortcut way to write a two-way if Statement (if-then-else). Consist of the System ? and :. Also called the ternary operator

Truth Table (!):

P	!p	example
T	F	!(age > 18) is false
F	T	!(age > 18) is True

Truth table (||):

P_1	P_2	$P_1 \mid\mid P_2$
F	F	F
F	T	T
T	F	T
T	T	T

Swith Statement:

Swith Statement: Like an if-Statement, the Swith provides ability to Select a path of execution from a group of possibility

```
Swith (Statis) {
  Case 0: ← break;
  Case 1:
  default:
}
```

Rules: Must always be in parenthesis. Must evalute to a value of type char, String, byte. Short, int, long. The data type of the expression must math data type of value 1, value 2, ect. The break Statement is optional and ends a case.

Which One Should Be used - if or Switch:

Use an If Statement: Need to test for boolean conditions.

Use an Switch Statement: Situations where looking for a Constant value and the cases are Simple.

Conditional Expressions:

Purpose: Shortcut way to write a two-way if statement (if-then-else). Consist of the system ? and :. Also called the ternary operator

General form: expression can be either a boolean value or a statement that evalutes to a boolean value

```
result = expression? value1 : value2;
```

4- Chapter 4

Peek into Objects:

Class: A container for one or more methods.

Method: Contains statements that performs a task.

objects: Represents an entity like a car, house, circle, dog, ect.

Constructors:

A Constructors is a method that is called to create an object.

Trigonometric Methods:

sin (double a)	acos (double a)	Math.sin(0)
cos (double a)	asin (double a)	returns 0.0
tan (double a)	atan(double a)	

Exponents Methods:

exp(double a)	Returns e raised to the power of a
log (double a)	Returns the natural log of a
log10 (double a)	Returns the 10-based log of a
pow (double a, double b)	Returns a raised to the power of b
Sqrt (double a)	Returns the square root of a

Rounding Methods:

double ceil (double x) x rounded up to its nearest integer. Returned as a double value.

double floor (double x) x rounded down to its nearest integer, returned as a double value.

double rint (double x) x rounded to its nearest integer. If x equally close to two integers, the even one is returned as a double

int round (float x)

long round (double x)

min, max, and abs

max (a,b) and min (a,b): Returns the max or min of two parameters

abs (a): Returns the absolute values of the parameters

random (): Returns a random double value in the range [0.0, 1.0).

Simple Methods for String objects:

Method	Description
length()	Returns the number of characters in the String
charAt(index)	Returns the character at the specified index from this string
concat(s1)	Returns a new String that concatenates this string with String s
toUpperCase()	Returns a new String with all letters in UpperCase
toLowerCase()	Returns a new String with all letters in lowerCase
trim()	Returns a new String with whitespace character trimmed on both s

Methods in the Character Class:

Methods	Description
isDigit(ch)	Returns true if the specified Character is a digit.
isLetter(ch)	Returns true if the specified Character is a letter.
isLetterOrDigit(ch)	Returns true if the specified character is a letter or digit
isLowerCase(ch)	Returns true if the specified character is a lowercase letter
isUpperCase(ch)	Returns true if the Specified Character is a uppercase letter
toLowerCase(ch)	Returns the lowercase of the specified character
toUpperCase(ch)	Returns the uppercase of the specified character

Char data type:

Data Type: Tells the complier what type of data is stored in a variable. Integer variables: byte, short, int, long. Floating-point variables: float, double. Boolean variables: boolean (values are true, false

```
//Checking for Uppercase Letter
if ((x >= 65) && (x <= 90)) {
```

The unicode is what is used in the comparison

```
String myString = "Silly Willy";
System.out.println("The length of " + myString + "is " + message.length
```

Reading Characters:

The Scanner DOES NOT have a nextChar() method.

Two Method are needed to read a character:

1. First use the Scanner's - next() or nextLine() method
2. Then use String's - charAt().

Shortcut way to write this:

```
System.out.print("Enter one character - any character*!:");
Char someChar = input.next().charAt(0);
System.out.println("Character entered = " + someChar);
```

Comparing strings:

Methods	Description
equals (s1)	Return true if this String is equal to String s1.
equalsIgnoreCase (s1)	~~Return true if~~ Same as equal, but it is case insensitive
compareTo (s1)	Returns (true) an integer ≥ 0, = 0, ≤ 0 to indicate whether this String is
compareToIgnoreCase(s1)	Same as compareTo, ~~but~~ except that the compare is case insensiti
StartsWith (prefix)	Returns true if the String starts with the specified prefix
endsWith(suffix)	Returns true if the ~~String~~ ~~starts~~ (end) with the specified suffix

~~Obtaining Substrings:~~

~~Methods~~
~~substring (beginIndex)~~

Finding a Character or a SubString in a String:

Method	Description (Returns the index of the ...)
indexOf (ch)	first occurence of ch in the String, else Return -1
indexOf (ch, (from) index)	first occurence of ch after fromIndex in the String, else return
indexOf (s)	first occurence of S in String, else return -1
indexOf (s, fromIndex)	first occurence of s after fromIndex in the String, else retur
lastIndexOf (ch)	last occurence of ch in String, else return -1
lastIndexOf(ch, fromIndex)	last occurence of ch after fromIndex in the String, else ret
lastIndexOf (s)	last occurence of s in String, else return -1
lastIndexOf (s, fromIndex)	last occurence of s after fromIndex in the String, else ret

5 - Chapter 5

Terminology:

Loop Body: Contains the statement that are repeated within the loop.

Loop iteration: One complete execution of the loop body

Infinite loop: A loop that runs forever.

Sentinel value: A value that signifies the end of a loop.

Pretest loop: A loop where the continuation condition is checked before the loop body is excuted.

Posttest loop: A loop where the continuation condition is checked after the loop body is excuted.

Coding Incrementally: A problem-solving approach.

How it works:

The loop codition - BE - is evaluted. If the condition is true, then the statement in the loop body are excuted.

Note: If the loop condition evalutes to false the 1st time, entire while loop is skipped.

Rules for Do while loop:

Theese are the same as while loop. The loop condition must be a boolean expression. Boolean Expressions must be in parentheses. Boolean expressions is formed using ~~expression~~ equality, relational and logical operators. Some statements within the loop body must eventually change the loop condition to false. If the condition is never changed to false, the program will be forever stuck in the loop. This is called an infinite loop. Curly brates are not necessary if only one statement in loop. Best practice is to always include curly braces.

Loop Design: Strategies for writting loops:

1st: figure out what part of your code needs to be repeated.

2nd: Place that part of your code into a loop.

3rd: ~~Make sure you have code in the loop body. Check~~ Determine what condition the loop checks.

4th: Make sure you have code in the loop body that eventually

Causes the condition to become false.

Terminating Loops:

Ask a question. Ask for the size of a list. Test for a particular value (sentintal value).

For loop:

Same purpose as while loop: a programming structure that performs repeated excuted (looping).

For loop are osed to execute a loop a preset number of times

How it works:

The control variable is initialized to a start value. The control variable is compared to the end value. If the test of the control variable evaluates to true then loop body is executed. The control variabe is updated after loop body completes.

Variable Scope:

Scope: the area of a program a variable can be referrenced.

N1: Declaring control variables before the for loop causes its scope to be inside and outside for loop.

N2: Declaring control variables in the ~~loop~~ for-loop causes its scope to be only inside the for loop.

Always use flags

flag = true / false

Frequently-Used Specifiers:

Specifier	output	Example
%b	a boolean value	True or False
%c	a character	'a'
%d	a decimal integer	200
%f	a floating-point number	45.460000
%e	a number in standard scientific notation	4.556000e+01
%s	a String	"Java is cool"

```
for ( ; ; ) {
  // Do Something
}
```

<---> equivalent <--->

```
while (true) {
  // Do Something
}
```


3

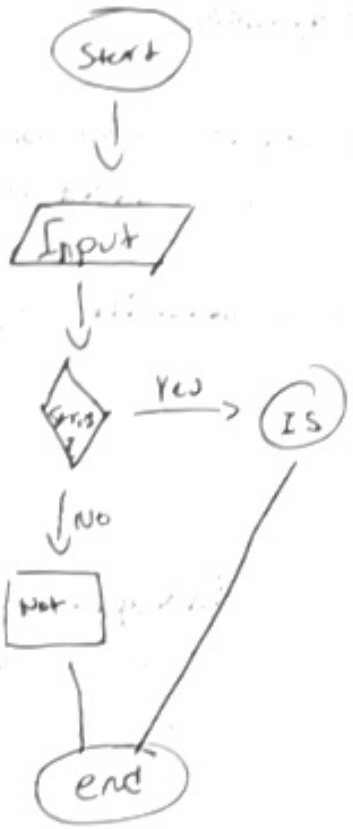
Breaks in loops:

Use a break in Switch Statements to end a case. Can be used in a loop to terminate a loop. Break out of loop. Breaking out of a while loop is a bad practice. Don't use breaks inside a loop in this case!

Continue in loops:

used to end current iteration of loop. Program control goes to end of loop body. Break out of current iteration. Breaking out of a for loop is a bad practice. Don't use breaks inside a loop in this case!

Can use breaks & continue in this class, just be careful & don't worry if your code looks ~~freaky deaky~~ weird.





6- Chapter 6

June

Terminology:

Method Signature: Combination of the method name and formal parameter list.

Modifiers: For methods the modifiers are the keywords public and Static

Method Name: A user defined name that describes the method

Formal Parameters: That value that are specified in the method definition. How information is passed to a method. Also called parameters. public Static Int Sum(int number)

Modular Programming: Breaking down of a problem into smaller, simpler, and more manageable parts and then coding those parts into independent units.

Methods: Group of Statements that perform a Specific task

Method Header: Specifies the modifiers, return type, method name, and parameters of a method.

Actual parameters: The value that are specified in the method invocation. Also called arguments. Sum(number);

Parameter lists: Can be formal parameter lists or actual parameter list

Return value Type: The data type of the value the method returns

Invoking a method: Calling a method with the actual parameters

Value Returning Method: A method that returns a value.

Void method: A method that performs some tasks without returning a value.

Overloading: Two or more methods with the same name but different formal parameters

Why Methods:

Provides a way to organize and simplify code. Place code that is used over and over into a method. Once code is in a method, it can be called many time. Improves code quality. Breaking code into units makes it easier to plan, code, test, and modify.

Method Definition:

When you define a method, you are defining, what value it returns (if any). What value it takes (if any) What the method does. A method is a collection of statements that are grouped together to perform an operation

Public: method's access is public. This means the method is visible to all classes

Static: Static methods belong to a class. For now make methods static, will discuss more with objects.

return Type: data type for value that is returned (int, double, boolean, ect.) or void.

methodName: a user defined name

formal parameter list: data that is coming into the method.

Method Signature: The combination of the method name and the parameter list.

Formal Parameters: The variables defined in the method header

Rules for Methods: A method may or may not return a value. A method must declare a return type. The number, order, and type of values passed in must match the number, order, and type of the parameters declared in the methods.

Void Methods:

Does not return a value to the caller. The call to a void method must be made as a statement

Notes:

The value returned by the method should be stored in a variable by the code that called the method

Call Stack:

The call stack is used to keep track of active methods. A stack is a data structure in Computer Science

Rules:

If a method declares formal parameters, you must pass something

and the values must match in type. The name of the actual and formal parameter may be the same or different: it does not matter. The actual parameters must match the formal parameter in order, number, compatible type

Overloading Methods:

When you have two or more methods with the same name but different formal parameter (signature). Gives you the ability to create multiple versions of a method

Rules: To be considered an overload method: Name (must be the same), Return Type (can be different: but cannot be the only thing you), Formal parameter (must be different)

Ambiguous Invocation

Sometimes there maybe two or more possible matches for an invocation of a method; but the compiler determine the most specific match. This is referred to as ambigous invocation. Ambiguous invocation is a compile error.

Scope of Local variables:

A local variable: a variable defined inside a method.

Scope: The part of ~~or local~~ the program where the variable can be referenced.

Note: Formal parameter are considered local variables. A variable declared in the initial action part of a for loop header has its Scope in the entire loop. But a variable declared inside a for loop body has its scope limited in the loop body from its declaration and to the end of the block that contains the variable.

Modularize Code:

Helps with writing code incrementally. Improves quality. Easier to maintain. Easier to debug.

Caution with Returning Methods:

Recall: a return statement is required with a value-returning method.

Benefits of Methods:

Write a method once and reuse it anywhere. Information hiding. Hide the implementation from the user. Reduce complexity.

Stepwise Refinement (optional):

The concept of method abstraction be applied to the process of development programs. When writing a large program, you can use the divide and conquer strategy, also known as stepwise refinement, to decompose it into subproblems. The subproblems can be further decomposed into smaller, more manageable problems.

Benefits of Stepwise Refinement:

Simpler Program, Reusing methods, Easier developing, Debugging, and Testing. Better facilitating teamwork

7 - Chapter 7

July

Terminology:

Array: A data structure used to store a collection of values that are all the same type

Index: Refers to a specific ~~type~~ elements within an array. Must be an integer expression.

Indexed Variable: Used to reference each element in an array.

Array Initializer: A single statement that declares, creates, and initializes an array

Out of Bounds error: Attemping to access an element at an index that is outside the range of the array. Causes a runtime error.

Off by One Error: Mistakenly referencing the first element in an array with the index 1 not index 0, or the last element with an ~~array~~ index that is one greater then the range of the array.

Garbage Collection: When the Java Virtual Machine (JVM) determines that memory used by an object can no longer be reached or used by the program and deallocates the memory (so it can be used again)

linear Search: Searching an array for a specific key value by starting at beginning and comparing the key with each and every element sequentially.

Binary Search: Searching a pre-sorted array by spillitting the array in half on each comparison.

Selection Sort: Combination of Searching and sorting. During each pass, the smallest element is moved to its proper location in the array

Arrays:

Array is a data structure that represents a collection of the same data type.

Why use Arrays: Provides ability to store a fixed set of values of the same type. Can access that set of values though one variable name.

Creating an Arrays: General form for creating an array:

arrayReferenceVariable = new dataType[arraySize]

This Statement does 2 things. creates an array with arraySize elements. Assigns to the reference variable arrayReferenceVariable the address of the array.

Example:

int[] numberList; → int[] numberList = new int[5]

numberList = new int[];

Array Defaultt value: Unlike other type, when an array of primitives is created, it is filled with default values. The default value is based on the element type the array will store. Zero, ifbor numeric. /U0000 - for char type - this is Unicode for "null". False for boolean type

Accessing Array Elements: Arrays are Zero based -just like strings

Example of Processing Arrays:

Foreach loops: Shortcut Java provides. Sequentially process loop without starting an index

Array Rules: Once created, an array's size is fixed while the program executes. All elements in an array will be the same type.

Notes: The declaration of an array can be written as double[] myArray; or was double myArray[]; The 1st approach is the preferred approach

Duplicating Arrays: With primitives it was easy to copy a value // Copy primitives:

```
int num1 = 10;
int num2 = num1;
```

Copy an Arrays: Use a loop to copy individual elements one by one:

// Correct way to copy arrays:

```
for (int i=0; i < array1.length; i++) {
    array2[i] = array1[i];
}
```

Use the static arraycopy method in the System Class:

```
System.arraycopy(array1, 0, array2, 0, array1.length);
```

use the clone method to copy arrays:

Passing Arrays: When passing arrays, you Do NOT use array notation in the actual parameter list. Correct: Swap (num1, num2); Incorrect: Swap (num1[], n
or. Swap (num1[10], num2[10]);

```
// Array initializer - declares, creates, initialize in one statement
 int[] numberList = {3,34,44,81,73};
 for (int i=0; i < numberList.length; i++){
   System.out.println ("numberList[" + i + "] = " + numberList[i]);
 }
```

Passing Primitives vs. Passing Arrays: Primitives types - Java uses pass by value. Arrays - Java uses pass by references

Pass by Value:

Means a copy of the value/variable is sent to the method. The orginal variable outside method is not changed by anything that happened inside the method.

```
public static void main (String[] args) {
  int num1 = 7;
  int num2 = 82;
Swap(num1, num2);
}// main
public static void Swap (int num1, int num2) {
  //swap the values
  int temp = num1;
  num1 = num2;
  num2 = temp;
}// Swap
```

Pass by References:

A reference to the array is sent to the method. This means the method now has the address of the array on the heap so any changes it makes to the array contents are contents are occuring to the "actual" array, not a copy of the array. The orginal array outside the method is affected by changes inside the method

Array Class:

Just like the Math Class was different, so is the Array class. We don't create a Math object, and we don't create an Array object. This is possible because ALL methods in the Math & Array class are what we call Static methods. Static methods allow you to call the methods in a class without creating an object of the class.

Searching: In computer programming a common activity is to write code to find a specific value in an array. Two techniques we will look at are: linear search, binary search.

Linear Search: Start at beginning of a list and look for a specific value one element at a time. When a match is found the index of the match is returned, otherwise a -1 is returned.

Linear Search example:

Key	List
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8

math.pow (2, int) power of 2

(Math.random * 20) + 1; math.random

7 - Chapter 7

Jubg

Stack and heap:

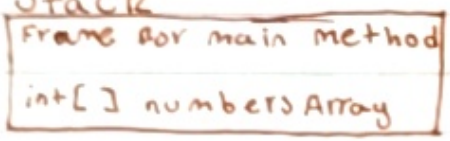
Java divides memory into 2 areas.

Stack memory: Stores local variables, reference variables, and method innovations. Also known as call stack.

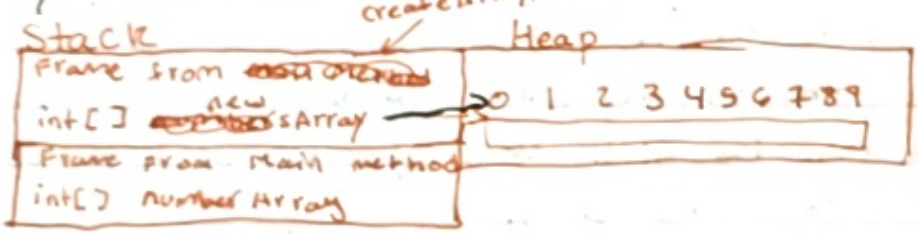
Heap memory: Stores objects (area that's garbage collection).

Integer array is declared: We now have a reference variable that is not yet referring to memory on the heap.

Stack Example: `int[] numbersArray = createArray(10);`



Heap Example: Mains calls createArray - a frame for createArray is pushed onto stack. Memory is allocated on the heap for newArray inside createArray method. newArray is a reference to that memory. newArray references is returned from that method, that is, an address to array is returned



Indexed Variables:

The array elements are accessed through the index. The array indices are 0-based. Each element in the array is represented using the following syntax, know as an indexed variable: arrayRefVar[index]

Using indexed variables: After an array is created, an indexed variable can be used in the same way as a regular variable. For example, the following code adds the value in myList[0] and myList[1] to myList[2].

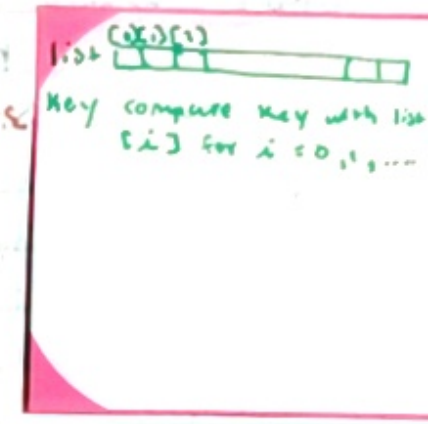
myList[2] = myList[0] + myList[1]

Searching Arrays: self Study

Searching Arrays: Searching is the process of looking for a specific element in an array; for example, discovering whether a certain score is

included in a list of scores. In this section, two commonly used approaches are discussed, linear search & binary search.

```
public class LinearSearch {
  // The method for finding a key in the list
  public static int linearSearch(int[] list, int key) {
    for (int i = 0; i < list.length; i++) {
      if (key == list[i])
        return i;
    return -1;
  }
}
```



Linear Search: The linear search approach compares the key element, key, sequentially with each element in the array list. The method continues to do so until the key matches an element in the list or the list is exhausted without a match being found. If a match is made, the linear search returns the index of the element in the array that matches the key. If no match is found, the search returns -1.

Binary Search: Consider the following three cases.

- If the key is less than the middle element, you only need to search the key in the 1st half of the array.
- If the key is equal to the middle element, the search ends with a match.
- If the key is greater than the middle element, you only need to search the key in the second half of the array.

Key	List								
8	1	2	3	4	5	6	7	8	9
8	1	2	3	4	5	6	7	8	9
8	1	2	3	4	5	6	7	8	9

Sorting Arrays:

Sorting, like searching, is also a common task in computer programming. Many different algorithms have been developed for

Sorting.

Selection Sort: Finds the smallest number in the list & places it first. It repeats this process until the list contains only a single number

Problem: Deck of Cards (pg 139 or Slide 7)

This problem builds a foundations for future more interesting & realistic applications.

ex
```
public class VarArgsDemo {
  public static void main (String[] Args) {
    printMax (34,3,3,2,56.5);
    printMax(new double[]{1,2,3});
  }

  public static void printMax (double... numbers) {
    if (numbers.length == 0){
      System.out.println("No argument passed");
      return;
    }
    double result = numbers[0];
    for (int i = 0; i < numbers.length; i++){
      if (numbers[i] > result)
        result = numbers[i];
    System.out.println("The max value is " + result);
  }
}//Example 6.10 pg 269 of textbook
```

Pass variable-length args list pass an array arg

A variable-length arg parameter

Line 3 invokes the printMax method with a variable length argument list passed to the array numbers. If no argument are passed, the length of the array is 0 (line 8). Line 4 invokes the printMax method with an array.